

Coding Challenge: Musik-Nutzungsdaten-Pipeline-Architektur

Dauer: 60 Minuten

Sprache: Java oder Python

Fokus: Systemdesign, Data Engineering, Architekturverständnis

Deine Aufgabe

Entwirf und implementiere eine **Datenverarbeitungs-Pipeline**. Die Abgabe erfolgt als GitHub Repo mit ausführbarem Code & Dokumentation. Die geschätzte Dauer ist nur ein Richtwert.

Szenario: Radiosender-Batch-Verarbeitung

Kontext:

Die GEMA verarbeitet täglich Millionen von Musik-Nutzungsereignissen aus verschiedenen Quellen. Diese müssen eingelesen, angereichert und transformiert werden damit nachfolgende Systeme basierend darauf die Ausschüttung der Lizenzentnahmen vornehmen können.

Szenario:

Ein Radiosender sendet täglich CSV-Dateien mit ihren Abspiellogs:

Input CSV-Format:

```
timestamp,src_code,station_id,duration_seconds,listener_count
2026-01-15T08:30:00Z,USRC17607839,RADIO_WDR,245,150000
2026-01-15T08:35:00Z,DEFR32400001,RADIO_WDR,180,150000
```

Felder:

- **timestamp:** ISO 8601 Format, wann das Lied gespielt wurde
- **src_code:** International Standard Recording Code (eindeutiger Code für das Musikwerk)
- **station_id:** Eindeutige ID des Radiosenders
- **duration_seconds:** Spieldauer des Werks in Sekunden
- **listener_count:** Geschätzte oder gemessene Höreranzahl während dieser Sendung

Anforderungen

1. Import der Datei

2. Verarbeitung der Datensätze

- Validierung / syntaktische Prüfung
- Anreicherung mit Werkdetails über eine Work-Catalog-API, z.B.: ISRC-Code → Werkdetails mit Künstler, Titel, Rechteinhaber
- Transformation (Berechnung relevanter Metriken: `listened_seconds = duration_seconds * listener_count`)

3. Speicherung verarbeiteter Datensätze

Berücksichtige:

- **Skalierbarkeit:** Dateien können 100MB+ mit Millionen von Datensätzen sein
- **Zuverlässigkeit:** Fehlgeschlagene Datensätze sollten wiederholbar sein
- **Observability:** Fehler müssen nachvollziehbar sein

Implementierung

Implementiere die Datenverarbeitung unter Verwendung einer Datenbank. Es muss nicht alles bis zu Ende fertig entwickelt sein, es ist wichtig, dass die Konzepte und Ideen erkennbar sind. Auch eine teilweise Umsetzung ist in Ordnung.

Schreibe den Code so, wie du ihn gerne lesen würdest.

Schreibe außerdem 2-3 Unit-Tests, die du für sinnvoll hältst, um die Datenverarbeitung zu testen.

Dokumentation

Schreibe eine kurze Doku mit folgenden Punkten:

- Architekturbeschreibung (gerne ein Diagramm)
- Welche Konzepte hast du umgesetzt (bzw würdest du umsetzen) um die Verarbeitung effizient zu machen
- Wie hast du Fehlerbehandlung umgesetzt
- Wie würdest du mit duplizierten Werken umgehen (fachlich und technisch)

Viel Spaß 😊